CREATE TABLE AdminDocs (

id int primary key,

xDoc Xml not null

)

- **Creates table AdminDocs.**
- **id = unique identifier (Primary Key).**
- **xDoc = stores XML data (untyped XML).**

INSERT INTO AdminDocs VALUES (6, '<catalog> ... </catalog>')

- **Insert XML catalog of products into row with id=6.**

INSERT INTO AdminDocs VALUES (2, '<doc id="123"> ... </doc>')

- **Insert another XML document with doc and sections into row with id=2.**

Basic Selection

select * from AdminDocs

- **View all rows (id + XML data).**

Using query() method

SELECT id, xDoc.query('/catalog') FROM AdminDocs

- **Extract the <catalog> root element.**

SELECT id, xDoc.query('/catalog/product') FROM AdminDocs

- **Extract direct <product> children of <catalog>.**

SELECT id, xDoc.query('//product') FROM AdminDocs

- **Extract all <product> nodes anywhere (descendant).**

SELECT id, xDoc.query('/*/product[@dept="WMN"]') FROM AdminDocs

- **Get products where attribute dept="WMN".**

SELECT id, xDoc.query('//product[number > 500]') FROM AdminDocs where id=6

- **Get all products where <number> value > 500.**

SELECT id, xDoc.query('/catalog/product[4]') FROM AdminDocs where id=6

- **Select 4th product element.**

SELECT id, xDoc.query('//product[number > 500][@dept="ACC"]') FROM AdminDocs where id=6

- **Products with number > 500 and dept="ACC".**

SELECT id, xDoc.query('//product[number > 500][1]') FROM AdminDocs where id=6

- **Only the first product satisfying condition.**

XQuery FLWOR Examples

SELECT xDoc.query('for $prod in //product let $x:=$prod/number return $x') FROM AdminDocs where id=6

- **Loops products → returns <number> values.**

SELECT xDoc.query('for $prod in //product let $x:=$prod/number where $x>500 return $x') FROM AdminDocs where id=6

- **Same but only numbers > 500.**

SELECT xDoc.query('for $prod in //product let $x:=$prod/number where $x>500 return (<Item>{$x}</Item>)') FROM AdminDocs where id=6

- **Wraps results inside <Item> tags.**

SELECT xDoc.query('for $prod in //product let $x:=$prod/number return if ($x>500) then <book>{$x}</book> else <paper>{$x}</paper>') FROM AdminDocs where id=6

- **Conditional return: if number>500 → <book>, else <paper>.**

Using exist()

SELECT id FROM AdminDocs WHERE xDoc.exist ('/doc[@id = 123]') = 1

- **Checks if <doc> with attribute id=123 exists → returns matching rows.**

Using value()

SELECT xDoc.value('(/doc//section[@num = 3]/title)[1]', 'varchar(100)') FROM AdminDocs

- **Extracts string value of <title> in section num=3.**

Modify XML

UPDATE AdminDocs SET xDoc.modify('insert <section num="2"><title>Background</title></section> after (/doc//section[@num=1])[1]')

- **Inserts a new section after section 1 in document.**

UPDATE AdminDocs SET xDoc.modify('delete //section[@num="2"]')

- **Deletes section with num=2.**

✅ **Summary for viva:**

- **query() → Extract XML nodes.**

- **exist() → Boolean check.**

- **value() → Extract scalar value.**

- **modify() → Insert/Delete/Update inside XML.**

- **// = anywhere (descendant), / = direct child.**

- **[@attr="val"] = filter by attribute.**

- **FLWOR (for-let-where-order-return) → loop with conditions, construct output.**

- **SELECT → tells SQL what you want to see (columns, results, or XML fragments).**

- **FROM → tells SQL which table the data comes from (AdminDocs).**

- **WHERE → adds conditions to filter rows (e.g., id=6).**

- **xDoc.query(...) → extract part of the XML.**

- **xDoc.exist(...) → check if a node/attribute exists (true/false).**

- **xDoc.value(...) → return a single scalar value from XML (string, int, etc.).**

- **xDoc.modify(...) → update the XML (insert, delete, replace).**

👉 **In short:**

- **SELECT = output**
- **FROM = source**
- **WHERE = filter**
- **query() = read XML nodes**
- **exist() = check XML condition**
- **value() = get one value**
- **modify() = change XML**

<catalog> is just the root element of your XML document.